

Java - Generici

doc. dr Vladimir Jocović

01.06.2026.

Motivacija za razvoj

- Generici u programskom jeziku Java omogućavaju ponovnu upotrebu koda.
- Reupotreba koda se postiže tako što se tipovima parametrizuju:
 - klase
 - interfejsi
 - metodi
- Na taj način moguća je realizacija generičkih (razlika samo u tipu):
 - zbirke (liste, skupovi, mape itd.)
 - algoritama i procedura

Pre uvođenja generika (Java 5)

- Kolekcije kao što je ArrayList čuvale su elemente kao Object.
- Referenca na Object može da veže podatak proizvoljnog tipa.
- Nije postojala tipna sigurnost (Type Safety).
- Bilo je neophodno ručno kastovanje (Cast).

```
ArrayList list = new ArrayList();
```

```
list.add("Hello");
```

```
list.add(123); // dozvoljeno
```

```
String s = (String) list.get(0); // cast is necessary, get returns Object
```

- Greške se otkrivaju tek u vreme izvršavanja programa.

Sa genericima u Javi

- Postoji tipna sigurnost (Type Safety) koju sprovodi prevodilac jakim proverama tipova u vreme prevođenja.
- Nije potrebno eksplicitno kastovanje (Cast).
- Greške se otkrivaju pri kompajliranju (mnogo lakše i brže nego u vreme izvršavanja).

```
ArrayList<String> list = new ArrayList<String>();  
list.add("Hello");  
// list.add(123); // compile time error  
String s = list.get(0); // does not need casting
```

Jednostavna kontejnerska klasa

```
public class Box {  
    private Object object;  
    public void set(Object object) { this.object = object; }  
    public Object get() { return object; }  
}
```

- Može da čuva objekte bilo kog tipa (referenca tipa Object).
- Nema načina da se u vreme prevođenja utvrdi sa objektima kog tipa se radi.
- Jedan deo koda može da očekuje da Box radi sa npr. stringovima i da ih postavlja, a drugi deo koda da Box radi sa celim brojevima i da ih dohvata, što rezultuje u grešci u vreme izvršavanja (java.lang.ClassCastException).

```
Box box = new Box();  
box.set("Hello");  
Integer num = (Integer) box.get(); // runtime error ClassCastException
```

Generička klasa

- Generička klasa se definiše u sledećem formatu:

```
class Name<T1, T2, ..., Tn> { /* ... */ }
```

gde je Name ime klase, a T1, T2, ..., Tn parametri tipova (type parameters).

```
public class Box<T> {  
    private T t;  
    public void set(T t) { this.t = t; }  
    public T get() { return t; }  
}
```

- T, kao parametar tipa, može da se koristi bilo gde unutar klase.
- T može da se zameni bilo kojim konkretnim neprimitivnim tipom prilikom kreiranja objekta generika: klasnim, interfejsnim, nizovskim ili drugim generičkim tipom (parametrizovanim tipom).

Konvencija imenovanja parametara tipova

- Uglavnom se koristi jedno veliko slovo.
 - Za razliku od imenovanja promenljivih: gear, gearRatio;
kao i konstanti: static final int GEAR_RATIO = 5;
- Često korišćena imena parametara tipova:
 - E – element kolekcije
 - K – ključ
 - N – broj (number)
 - T – tip
 - V – vrednost
 - S, U, V – drugi, treći i četvrti parametar tipa koji se navode.

Parametrizacija i instanciranje generičkog tipa

- Parametrizacija generičkog tipa (ne kreira objekat, već samo zamenjuje parametar generika T stvarnim argument tipom):

```
Box<Integer> integerBox; // T se menja Integer tipom
```

- Instanciranje generičkog tipa:

```
Box<Integer> integerBox = new Box<Integer>();
```

- Od Jave 7 nije neophodno navođenje argumenta tipa prilikom pozivanja konstruktora generičke klase u situacijama u kojima prevodilac može da zaključi argument iz konteksta (<> dijamant).

```
Box<Integer> integerBox = new Box<>();
```

Višestruki parametri tipa

- Generička klasa može da sadrži više parametara tipova:

```
public class Pair<K, V> {  
    private K key;  
    private V value;  
    public Pair(K key, V value) { this.key = key; this.value = value; }  
    public K getKey() { return key; }  
    public V getValue() { return value; }  
}  
  
// autoboxing int argument 7 to Integer  
Pair<String, Integer> p1 = new Pair<String, Integer>("Seven", 7);  
Pair<String, String> p2 = new Pair<>("Two", "2"); // Java diamond inference mechanism  
Pair<String, Box<Integer>> p3 = new Pair<>("even", new Box<>()); // parametrized type argument
```

Sirovi tipovi (Raw Types)

- Sirovi tip predstavlja ime generičkog tipa u kome nije naveden stvarni argument tipa koji menja parametar tipa.

```
public class Box<T> {  
    // ...  
}
```

- Parametrizovan tip: `Box<Integer> intBox = new Box<>();`
- Sirovi tip: `Box rawBox = new Box();`
 - Ovakav tip `Box` je sirovi tip generičkog tipa `Box<T>`.
- Negenerički tip nije sirovi tip (npr. ranije napisana realizacija `Box` klase sa `Object` referencom).

Sirovi tipovi (Raw Types)

- Sirovi tipovi se često pojavljuju u starijim delovima koda pisanim u ranijim verzijama Java (Legacy Code).
- Tadašnje realizacije većeg broja standardnih Java klasa nisu bile generičke (npr. Collections).
- Takve klase su držale reference na tip Object.
- Korišćenjem sirovih tipova ostvaruje se ponašanje kao pre uvođenja generika, koji su uvedeni u Javi 5.
- Sirovi tip podrazumeva postojanje istoimenog parametrizovanog (generičkog) tipa.

Sirovi tipovi (Raw Types)

- Pre Java 5 kod bi izgledao ovako:

```
Box box = new Box();
```

Nakon uvođenja generika kod izgleda ovako:

```
Box<String> box = new Box<>();
```

- Radi kompatibilnosti sa kodom pisanim u ranijim verzijama (da se ne bi „slomio“ kod napisan u prethodnim verzijama), dozvoljeno je dodeljivanje parametrizovanog tipa sirovom tipu:

```
Box<String> newBox = new Box<>();
```

```
oldMethod(newBox);
```

```
// oldMethod is legacy code which used to receive raw Box
```

Sirovi tipovi (Raw Types)

- Prethodna konverzija ne proizvodi ni upozorenje od strane prevodioca jer programer ništa dodatno ne tvrdi:

```
Box<String> stringBox = new Box<>();
```

```
Box rawBox = stringBox; // this is ok, no compiler warning
```

```
rawBox.set(123); // warning: unchecked invocation to generic set(T)
```

```
String s = stringBox.get(); // runtime error: ClassCastException
```

- Međutim, dodela sirovog tipa parametrizovanom tipu biće praćena upozorenjem od strane prevodioca (programer tvrdi tip podataka dodelom sirovog tipa):

```
Box rawBox = new Box(); // rawBox is a raw type of Box<T>
```

```
...
```

```
Box<Integer> intBox = rawBox; // warning: unchecked conversion
```

```
Integer x = intBox.get(); // runtime error: ClassCastException
```

Sirovi tipovi (Raw Types)

- Generici su implementirani pomoću Type Erasure mehanizma i zbog toga u vreme izvršavanja ne postoje informacije o parametrima tipa.
 - U vreme izvršavanja tipovi: `Box`, `Box<Integer>`, `Box<String>` su identični.
- Sirovi tipovi nemaju informaciju o generičkom tipu parametra ni u vreme prevođenja (nikad nisu imali):
 - Omogućavaju pojavu grešaka u vreme izvršavanja (`ClassCastException`).
 - Prevodilac ne može da izvrši proveru tipova za operacije koje se vrše nad njima, pa prijavljuje unchecked upozorenja.
- Stoga, zadržani su samo zbog kompatibilnosti sa programskim kodom napisanim u prethodnim verzijama (pre postojanja generika).

Generičke metode

- Generičke metode uvode sopstvene parametre tipova.
- Doseg tih parametara tipova je doseg tako deklarisanе metode.
- Nestatičke i statičke metode mogu biti generičke.
 - Parametri tipova pišu se ispred povratnog tipa metode.
- I konstruktori mogu biti generički.
 - Parametri tipova pišu se ispred imena konstruktora klase.

Generičke metode

```
public class Util {  
    public static <K, V> boolean compare(Pair<K, V> p1, Pair<K, V> p2) {  
        return p1.getKey().equals(p2.getKey()) &&  
            p1.getValue().equals(p2.getValue());  
    }  
}
```

```
Pair<Integer, String> p1 = new Pair<>(1, "apple");  
Pair<Integer, String> p2 = new Pair<>(2, "pear");  
boolean same = Util.<Integer, String>compare(p1, p2);
```

- Moguće je da prevodilac zaključi tipove parametara generika na osnovu stvarnih argumenata metode bez navođenja tipova argumenata unutar dijamanta (Type Inference), pa poziv generičke metode izgleda kao poziv obične metode.

```
boolean same = Util.compare(p1, p2);
```

Ograničeni parametri tipova (Bounded type parameters)

- Postoje situacije u kojima je potrebno ograničiti skup tipova koji se mogu koristiti kao argumenti generičkih parametara tipova.
 - Npr. metode koje rade nad brojevima treba da prihvate samo objekte klase Number ili tipova koji su izvedeni iz te klase.
- Takav ograničeni parametar se uvodi navođenjem najpre njegovog imena, iza kojeg sledi rezervisana reč extends, nakon čega sledi tip koji predstavlja gornju granicu.
 - Rezervisana reč extends koristi se i u slučaju klase i u slučaju interfejsa koji se navode kao tipovi koji predstavljaju gornju granicu.
- Greška je u prevođenju, ukoliko se kao argument tipa generika zada tip koji ne poštuje navedena ograničenja.

Ograničeni parametri tipova (Bounded type parameters)

```
public class Box<T> {  
    private T t;  
    public void set(T t) { this.t = t; }  
    public T get() { return t; }  
    public <U extends Number> void inspect(U u) {  
        System.out.println("T: " + t.getClass().getName());  
        System.out.println("U: " + u.getClass().getName());  
    }  
    public static void main(String[] args) {  
        Box<Integer> integerBox = new Box<Integer>();  
        integerBox.set(new Integer(10)); // ok, integerBox expects integer argument  
        integerBox.inspect("some text"); // compile-time error: String is not a subtype of Number  
    }  
}
```

Ograničeni parametri tipova (Bounded type parameters)

- Pored sprovođenja navedenih ograničenja tipova koji se mogu zadati kao argumenti tipa, moguće je i pozivanje metoda koji su deo interfejsa parametra tipa navedenog kao ograničenja.

```
public class NaturalNumber<T extends Integer> {  
    private T n;  
    public NaturalNumber(T n) { this.n = n; }  
    public boolean isEven() { return n.intValue() % 2 == 0; }  
    // ...  
}
```

- intValue je nestatički metod klase Integer.

Ograničeni parametri tipova (Bounded type parameters)

- Moguće je navesti više parametara tipova koji predstavljaju ograničenja i oni moraju da budu razdvojeni znakom &.
 - Parametar tipa generika mora da bude podtip svih navedenih parametara tipova ograničenja.
- Ukoliko je klasa jedan od tipova parametara koji predstavljaju ograničenja, ona mora biti navedena prva.
 - U suprotnom, dešava se greška u vreme prevođenja.

```
class A { /* ... */ }
```

```
interface B { /* ... */ }
```

```
interface C { /* ... */ }
```

```
// T is a subtype of all types listed below (A, B and C)
```

```
class D <T extends A & B & C> { /* ... */ } // ok, class A is first
```

```
class D <T extends B & A & C> { /* ... */ } // compile-time error
```

Generičke metode i ograničeni parametri tipova

- Razmatra se implementacija metode koja broji koliko elemenata niza je svojom vrednošću veće od zadate vrednosti:

```
public static <T> int countGreaterThan(T[] anArray, T elem) {  
    int count = 0;  
    for (T e : anArray)  
        if (e > elem) // compiler error  
            ++count;  
    return count;  
}
```

- Navedeni kod ne može da se prevede, jer operator > može da se primeni nad primitivnim tipovima (byte, char, int, float itd.), ali ne i nad parametrom tipa T.

Generičke metode i ograničeni parametri tipova

- Da bi operator > mogao da se primeni nad objektima neke klase, ta klasa mora da implementira interfejs Comparable<T>.

```
public interface Comparable<T> {  
    public int compareTo(T o); // returns >0 if this is greater than o, 0 if equal, <0 if lower  
}
```

- Primenom ograničenog parametra tipa kod prethodne metode izgleda ovako:

```
public static <T extends Comparable<T>> int countGreaterThan(T[] anArray, T elem) {  
    int count = 0;  
    for (T e : anArray)  
        if (e.compareTo(elem) > 0) // this is ok now  
            ++count;  
    return count;  
}
```

Generici, nasleđivanje i podtipovi

- Referenca tipa osnovne klase može da referiše objekte izvedene klase (subtype „is a“ supertype), ali obrnuto ne važi.

`Object someObj = new Integer(10); // ok, Integer is a subtype of Object`

`Integer someInt = someObj; // compile-time error: explicit cast required`

`public void someMethod(Number n) { /* ... */ }`

`someMethod(new Integer(10)); // ok, Integer is a subtype of Number`

- Isto važi i u slučaju pozivanja metoda generika:

`Box<Number> box = new Box<Number>();`

`box.set(new Integer(10)); // ok, since Integer is a subtype of Number`

Generici, nasleđivanje i podtipovi

- Razmatra se sledeći metod:

```
public void boxTest(Box<Number> n) { /* ... */ }
```

- Bilo bi za očekivati da stvarni argument tipa `Box<Integer>` ili `Box<Double>` može da se veže za formalni parametar `n` tipa `Box<Number>` metode `boxTest`, s obzirom da su tipovi `Integer` i `Double` podtipovi tipa `Number`.
- Međutim, to nije dozvoljeno jer tipovi `Box<Integer>` i `Box<Double>` nisu podtipovi tipa `Box<Number>`.

```
Box<Integer> intBox = new Box<>();
```

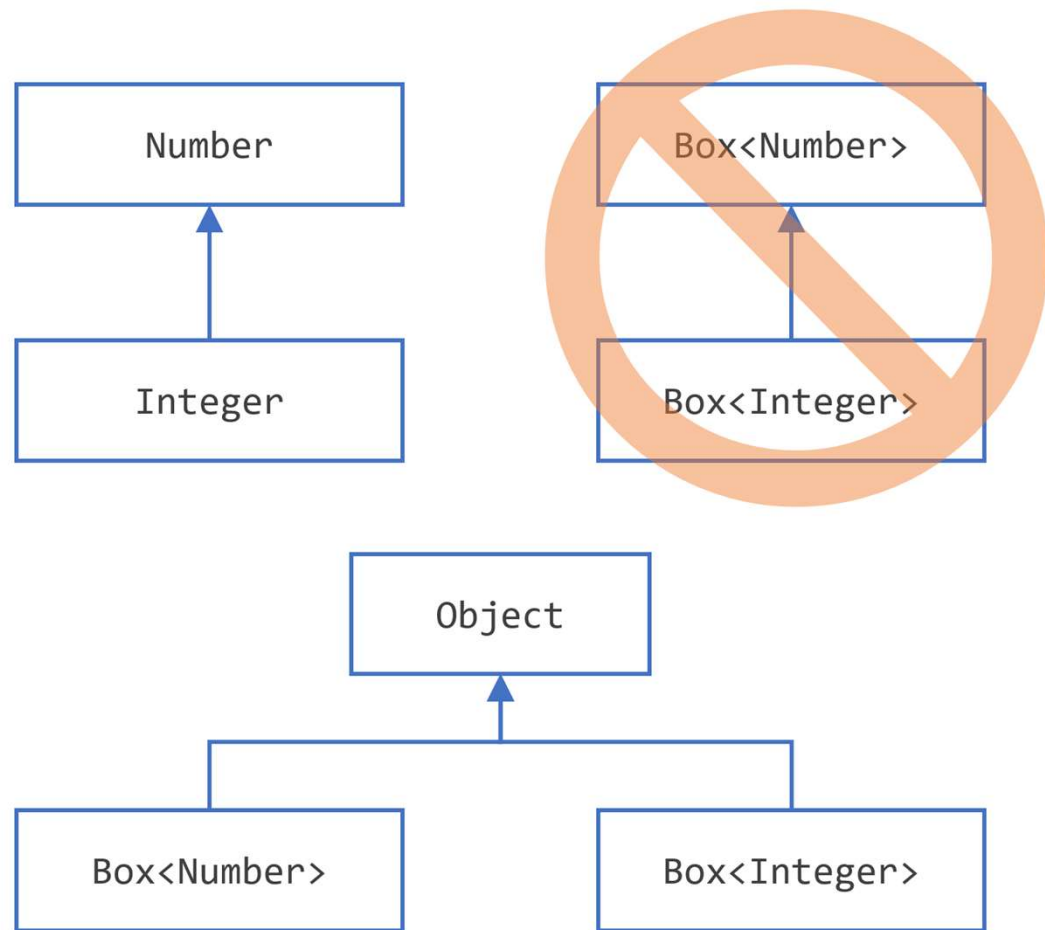
```
Box<Number> numBox = intBox; // compile-time error, but assume is allowed
```

```
numBox.set(3.14); // setting double value to intBox!
```

```
// allowed, since Double is subtype of Number
```

- To bi narušilo tipnu sigurnost koju uvode generici.

Generici, nasleđivanje i podtipovi



Generici i nasleđivanje

- Generičke klase i interfejsi mogu da se proširuju (nasleđuju).
- Npr. generička klasa `ArrayList<E>` implementira generički interfejs `List<E>`, a generički interfejs `List<E>` proširuje generički interfejs `Collection<E>`.
- Stoga, npr. tip `ArrayList<String>` je podtip tipa `List<String>`, a tip `List<String>` je podtip tipa `Collection<String>`.
- U slučaju da se ne varira stvarni argument parametar tipa, relacija nasleđivanja se zadržava među generičkim tipovima.

Generici i nasleđivanje

- Ukoliko postoji sledeći interfejs koji implementira interfejs List<E>:

```
interface PayloadList<E,P> extends List<E> {  
    void setPayload(int index, P val); // associates val with list element  
}
```

sledeći primeri tipova podataka jesu podtipovi tipa List<String>:

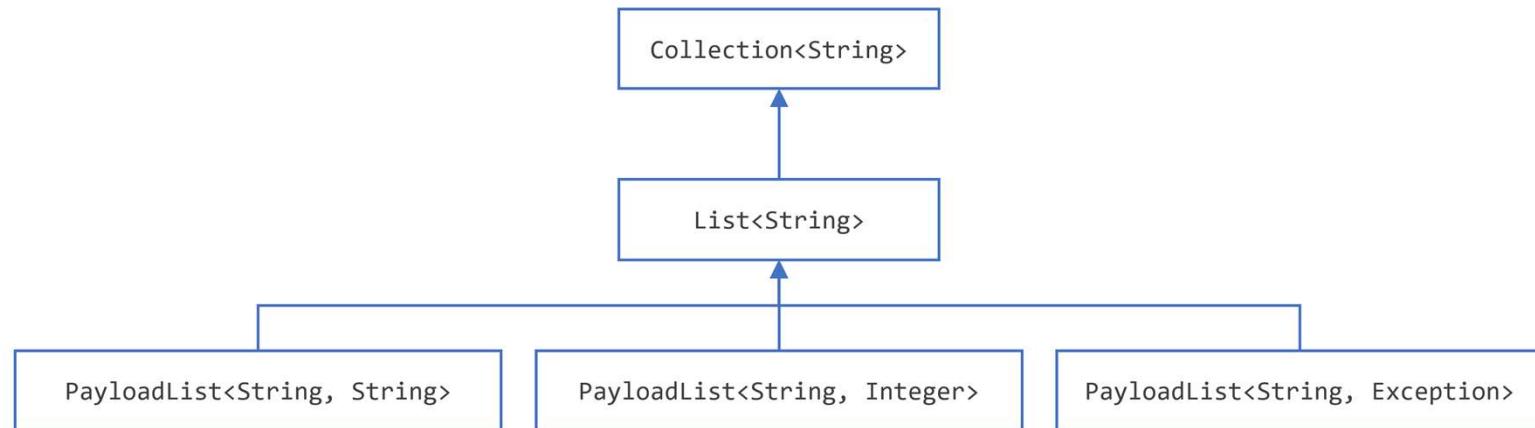
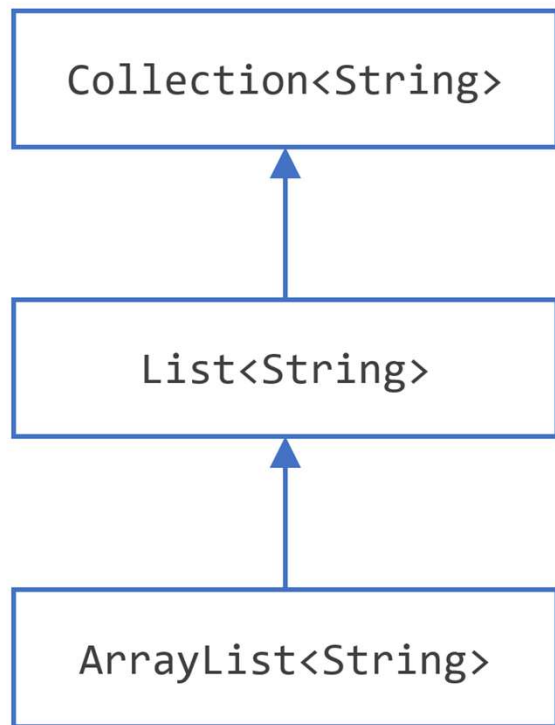
PayloadList<String, String>

PayloadList<String, Integer>

PayloadList<String, Exception>

- Svaki PayloadList<E, P> jeste podtip List<E>, bez obzira na to šta je P.

Generici i nasleđivanje



Zaključivanje tipa

- Prevodilac može da, na osnovu poziva metode, njene deklaracije i stvarnih argumenata, zaključi konkretne parametre tipova za koje je poziv metode moguć, kao i tip povratne vrednosti, ukoliko postoji.
- Algoritam zaključivanja pokušava da pronađe najspecifičniji, najpribližniji tip koji radi sa svim argumentima:

```
static <T> T pick(T a1, T a2) { return a2; }
```

```
Serializable s = pick("d", new ArrayList<String>());
```

- Kog tipa je T – String ili ArrayList<String>?
- Najbliži zajednički tip je Serializable (i String i ArrayList<String> implementiraju ovaj interfejs).
 - Object im je, takođe, zajednički tip, ali nije najpribližniji.
 - Kontekst dodele takođe pomaže u određivanju najpribližnijeg tipa.

Zaključivanje tipa

```
public class BoxDemo {  
    public static <U> void addBox(U u, java.util.List<Box<U>> boxes) {  
        Box<U> box = new Box<>(); box.set(u); boxes.add(box);  
    }  
    public static <U> void outputBoxes(java.util.List<Box<U>> boxes) {  
        int counter = 0;  
        for (Box<U> box: boxes) { System.out.println("Box #" + (++counter) + " contains [" + box.get().toString() + "]); }  
    }  
    public static void main(String[] args) {  
        java.util.ArrayList<Box<Integer>> listOfIntegerBoxes = new java.util.ArrayList<>();  
        BoxDemo.<Integer>addBox(Integer.valueOf(10), listOfIntegerBoxes); BoxDemo.addBox(Integer.valueOf(20), listOfIntegerBoxes);  
        BoxDemo.outputBoxes(listOfIntegerBoxes);  
    }  
}  
  
// Box #1 contains [10]  
// Box #2 contains [20]
```

Zaključivanje tipa i generički metodi

- Generički metod navodi parametar tipa nazvan U:

```
public static <U> void addBox(U u, java.util.List<Box<U>> boxes);
```

- Prevodilac može da zaključi parametar tipa na osnovu konteksta, pa se u većini slučajeva oni ne moraju eksplicitno navoditi.
- Pozivanje generičke metode addBox može da se uradi eksplicitnim navođenjem stvarnog argumenta parametra tipa (type witness, npr. <Integer>):

```
BoxDemo.<Integer>addBox(Integer.valueOf(10), listOfIntegerBoxes);
```

- Međutim, eksplicitno navođenje stvarnog argumenta parametra tipa može i da se izostavi:
 - Prevodilac automatski zaključuje na osnovu tipova argumenata metode addBox da je stvarni argument parametar tipa Integer.

```
BoxDemo.addBox(Integer.valueOf(20), listOfIntegerBoxes);
```

Zaključivanje tipa i instanciranje generika

- Prilikom pozivanja konstruktora generičke klase moguće je da se izostave stvarni argumenti parametar tipova, dokle god prevodilac može da ih zaključi na osnovu konteksta (<> dijamant).
- Umesto:

```
Map<String, List<String>> myMap = new HashMap<String, List<String>>();
```

može da se napiše:

```
Map<String, List<String>> myMap = new HashMap<>();
```

- Međutim, treba voditi računa da se dijamant ne izostavi:

```
Map<String, List<String>> myMap = new HashMap(); // unchecked conversion warning
```

- U tom slučaju kompajler generiše upozorenje, jer se `HashMap()` konstruktor odnosi na sirovi tip `HashMap`, a ne na tip `HashMap<String, List<String>>`.

Zaključivanje tipa i generički konstruktori

- Konstruktori mogu da budu generički (deklarišu svoje parametre tipova).
 - I u generičkim i u negeneričkim klasama.

```
class MyClass<X> { // type parameter X defined by class
    <T> MyClass(T t) {} // type parameter T defined by constructor
}
```

- Integer je eksplicitno naznačen stvarni argument tipa za parametar tipa X, dok je String eksplicitno naznačen stvarni argument tipa za parametar tipa T.

```
MyClass<Integer> obj1 = new <String> MyClass<Integer>("");
```

- Integer je eksplicitno naznačen stvarni argument tipa za parametar tipa X, dok prevodilac zaključuje da je za parametar tip T stvarni argument tipa String, s obzirom da je stvarni argument konstruktora tipa String (literal "").

```
MyClass<Integer> obj2 = new MyClass<Integer>("");
```

- U oba slučaja prevodilac zaključuje: da je za parametar tip X stvarni argument tipa Integer, a da je za parametar tip T stvarni argument tipa String.

```
MyClass<Integer> obj3 = new MyClass<>("");
```

Ciljni tipovi

- Prevodilac može da koristi i ciljni tip da utvrdi parametre tipova prilikom pozivanja generičke metode.
- Ciljni tip izraza je tip podatka koji prevodilac očekuje u zavisnosti od toga gde se izraz pojavljuje.

```
static <T> List<T> emptyList(); // Collections class method  
List<String> listOne = Collections.emptyList();
```

- Ciljni tip ove naredbe je List<String>, a s obzirom da je povratni tip metoda emptyList zapravo List<T>, prevodilac zaključuje da je za parametar tip T stvarni argument tip String.
- Eksplicitno navođenje stvarnog tip argumenta moguće je navođenjem svedoka tipa (type witness).

```
List<String> listOne = Collections.<String>emptyList();
```

Ciljni tipovi

```
void processStringList(List<String> stringList) { process(stringList); }
```

- U ranijim verzijama Jave (verzija 7), tip formalnog parametra se nije posmatrao kao ciljni tip pa se sledeći kod nije prevodio uspešno:

```
// Java7: List<Object> cannot be converted to List<String>  
processStringList(Collections.emptyList());
```

- Zbog toga je, u ovom kontekstu, navođenje svedoka tipa bilo obavezno:

```
processStringList(Collections<String>.emptyList());
```

- U međuvremenu, od Java verzije 8, proširen je kontekst postojanja ciljnog tipa tako da uključuje i formalne parametre metoda. Iz tog razloga, navođenje svedoka tipa više nije obavezno.

```
processStringList(Collections.emptyList()); // compiles ok in Java8
```

Ciljni tipovi u lambda izrazima

```
public interface CheckPerson { // functional interface
    boolean test(Person p);
}
```

```
public static void printPersons(List<Person> roster, CheckPerson tester);
printPersons(people, p -> p.getAge() >= 18 && p.getAge() <= 25);
```

- U metodu printPersons formalni parametar tester je tipa CheckPerson, pa je to i ciljni tip stvarnog argumenta lambda izraza.

```
public void printPersonsWithPredicate(List<Person> roster, Predicate<Person> tester);
printPersonsWithPredicate(people, p -> p.getGender() == Person.Sex.M);
```

- U metodu printPersonsWithPredicate formalni parametar tester je tipa Predicate<Person>, pa je to i ciljni tip stvarnog argumenta lambda izraza.

Ciljni tipovi u lambda izrazima

- Da bi odredio tip lambda izraza, prevodilac koristi ciljni tip konteksta u kome se taj lambda izraz našao.
- Lambda izraz može da se koristi u situacijama u kojima prevodilac može da odredi ciljni tip.

```
interface Predicate<T> {  
    boolean test(T t); // evaluates predicate on given argument  
}
```

Ciljni tipovi u lambda izrazima

- Deklaracija (definicija) promenljive:

```
Predicate<String> p = s -> s.isEmpty();
```

- Dodela vrednosti:

```
Predicate<String> p;  
p = s -> s.length() > 3;
```

- Naredba return:

```
Predicate<String> getPredicate() { return s -> s.startsWith("A"); }
```

- Inicijalizator niza:

```
Predicate<String>[] arr = new Predicate[] { s -> s.isEmpty(), s -> s.length() > 5 };
```

Ciljni tipovi u lambda izrazima

- Argument konstruktora ili metode:

```
List<String> list = new ArrayList<>(Arrays.asList("Ana", "Marko", "Ivana"));  
list.removeIf(s -> s.length() < 4);
```

- Telo drugog lambda izraza (ugnežđeni lambda izraz):

```
Function<String, Predicate<String>> f = str -> (s -> s.contains(str));
```

- Ternarni operator (?:):

```
String mode = "U";  
Predicate<String> p = "U".equals(mode) ? s -> s.equals(s.toUpperCase()) : s -> s.equals(s.toLowerCase());
```

- Operator konverzije (cast):

```
Predicate<String> p = (Predicate<String>) (s -> s.endsWith("a"));
```

Ciljni tipovi i argumenti metoda

- Za argumente metoda, prevodilac određuje ciljni tip pomoću 2 jezičke karakteristike:
 - Razrešavanje preopterećenih metoda (overload resolution).
 - Zaključivanje tipova argumenata (type argument inference).

```
public interface Runnable { void run(); } // returns void
```

```
public interface Callable<V> { V call(); } // returns V
```

- U slučaju da postoje dve definisane metode invoke:

```
void invoke(Runnable r) { r.run(); }
```

```
<T> T invoke(Callable<T> c) { return c.call(); }
```

- sledeći poziv metode invoke poziva invoke(Callable<T>) metod:

```
String s = invoke(() -> "done");
```

- s obzirom da metod invoke(Callable<T>) vraća vrednost, a invoke(Runnable r) ne.
- Ciljni tip lambda izraza () -> "done" je Callable<T>.

Wildcard (džoker) – gornje ograničen tip

- Šta, ukoliko je potrebno da neki metod radi nad tipovima `List<Integer>`, `List<Float>` i `List<Double>`?
- Rešenje nije napraviti samo jedan metod čiji je formalni parametar tipa `List<Number>`, jer taj tip nije supertip tipova `List<Integer>`, `List<Float>` i `List<Double>`, iako su `Integer`, `Float` i `Number` podtipovi tipa `Number`.
- Međutim, moguće je koristiti odozgo ograničeni džoker (upper bounded wildcard) radi ublažavanja prethodno navedene restrikcije.
- Gornje ograničeni džoker uvodi se pisanjem džoker znaka `?` koga prati rezervisana reč **`extends`** koju prati tip podataka koji predstavlja gornju granicu (extends se koristi i u slučaju klasa i u slučaju interfejsa).
- Džoker `?` može da se koristi samo u okviru generičkog tipa podataka.

Wildcard (džoker) – gornje ograničen tip

- Da bi se omogućilo postojanje jednog metoda koji može da radi sa listama podataka tipa Number, kao i listama podataka podtipa tipa Number (Integer, Float, Double itd.), odnosno da bi se omogućilo da metod pored tipa List<Number> može da prihvati i argumente tipa List<Integer>, List<Float> i List<Double>, neophodno je da se formalni parametar takvog metoda definiše kao List<? extends Number>.
- Formalni parametar tipa List<Number> je restriktivniji od formalnog parametra tipa List<? extends Number>, jer List<Number> može da prihvati samo liste čiji su podaci tipa Number, dok List<? extends Number> može da prihvati liste čiji su podaci tipa Number ili klasa izvedenih iz tipa Number.

Wildcard (džoker) – gornje ograničen tip

```
public static double sumOfList(List<? extends Number> list) {  
    double s = 0.0;  
    for (Number n : list)  
        s += n.doubleValue();  
    return s;  
}  
  
// Arrays.asList creates fixed-size list backed by array,  
// which elements can be replaced, but its size cannot be changed  
List<Integer> listInt = Arrays.asList(1, 2, 3);  
System.out.println("sum = " + sumOfList(listInt)); // prints: sum = 6.0  
List<Double> listDouble = Arrays.asList(1.2, 2.3, 3.5);  
System.out.println("sum = " + sumOfList(listDouble)); // prints: sum = 7.0
```

Wildcard (džoker) – gornje ograničen tip

```
public static void process(List<? extends Foo> list) {  
    for (Foo elem : list) {  
        // some processing ...  
    }  
}
```

- Metodu je moguće proslediti listu podataka tipa Foo ili listu podataka čiji su tipovi izvedeni iz tipa Foo.
- U okviru metoda se elementima liste pristupa kao da su tipa Foo.

Wildcard (džoker) – neograničen tip

- Definiše se navođenjem samo džoker znaka ?.
- `List<?>` predstavlja listu podataka nepoznatog tipa.
- Za bilo koji tip podataka `X`, `List<X>` je podtip tipa `List<?>`.
- S obzirom da tip podataka nije poznat, ovakav generički tip može da bude koristan u sledećim situacijama:
 - Ukoliko se piše kod koji se oslanja na funkcionalnosti koje su obezbeđene u `Object` klasi.
 - Ukoliko se piše kod koji koristi metode generičke klase koje ne zavise od parametar tipa (npr. `List.size()` ili `List.clear()`).

Wildcard (džoker) – neograničen tip

```
public static void printList(List<?> list) {  
    for (Object elem: list)  
        System.out.print(elem + " ");  
    System.out.println();  
}  
  
List<Integer> listInt = Arrays.asList(1, 2, 3);  
printList(listInt); // prints: 1 2 3\n  
List<String> listStr = Arrays.asList("one", "two", "three");  
printList(listStr); // prints: one two three\n
```

Wildcard (džoker) – neograničen tip

- Treba voditi računa da sledećem kodu, koji vrši ispisivanje elemenata liste, ne bi mogla da se prosledi lista podataka nekog drugog tipa, osim tipa Object.

```
public static void printList(List<Object> list) {  
    for (Object elem: list)  
        System.out.print(elem + " ");  
    System.out.println();  
}
```

- Iako su tipovi Integer, Double i String podtipovi tipa Object, tipovi List<Integer>, List<Double>, List<String> nisu podtipovi tipa List<Object>.

Wildcard (džoker) – neograničen tip

- Tipovi `List<Object>` i `List<?>` nisu isti tipovi podataka.
- U listu tipa `List<Object>` mogu da se dodaju podaci bilo kog tipa, jer je `Object` nadtip bilo kog tipa podataka.

```
List<Object> listObj = new ArrayList<>();
```

```
listObj.add(12); // ok, Object reference can hold Integer
```

```
// List<Object> is not a supertype of List<Double> or ArrayList<Double>
```

```
listObj = new ArrayList<Double>(); // compile-time error
```

```
listObj.add(3.14); // ok, Object reference can hold Double
```

- U listu tipa `List<?>` može da se doda samo vrednost **null** (ostale vrednosti bi narušile tipnu sigurnost).

```
List<?> list = new ArrayList<>();
```

```
list.add(null); // ok
```

```
list = new ArrayList<Integer>(); // ok
```

```
list.add(3.14); // compile-time error
```

Wildcard (džoker) – donje ograničen tip

- Gornje ograničeni džoker omogućava da stvarni parametar tip generika može da bude ili tipa navedenog gornjom granicom ili nekog od podtipova tog tipa gornje granice.
- Sa druge strane, donje ograničeni džoker omogućava da stvarni parametar tip generika bude ili tipa navedenog donjom granicom ili nekog od nadtipova tog tipa donje granice.
- Donje ograničeni džoker uvodi se pisanjem džoker znaka ? koga prati rezervisana reč **super** koju prati tip podataka koji predstavlja donju granicu (super se koristi i u slučaju klase i u slučaju interfejsa).
- Uz korišćenje džokera ? moguće je navesti ili samo gornju granicu tipa ili samo donju granicu tipa, ali ne i obe.

Wildcard (džoker) – donje ograničen tip

- Ukoliko se želi dodavanje objekata tipa Integer u listu, fleksibilno rešenje bi bilo da jedan metod može da dodaje takve objekte u bilo koju listu koja može da drži takve objekte.
- List<Integer>, List<Number> i List<Object> mogu da drže takve objekte, jer je tip Integer podtip tipova Number i Object.
- Da bi se definisao metod koji može da prihvata liste koje drže podatke tipa Integer ili njegovih nadtipova, formalni parametar takvog metoda treba da bude List<? super Integer>.
- Formalni parametar tipa List<Integer> je restriktivniji od formalnog parametra tipa List<? super Integer>, jer List<Integer> može da prihvati samo liste čiji su podaci tipa Integer, dok List<? super Integer> može da prihvati liste čiji su podaci tipa Integer ili nadtipova tipa Integer.

Wildcard (džoker) – donje ograničen tip

```
public static void addNumbers(List<? super Integer> list) {  
    for (int i = 1; i <= 10; i++) list.add(i);  
}  
List<Integer> listInt = new ArrayList<>();  
addNumbers(listInt); // ok  
List<Number> listNum = new ArrayList<>();  
addNumbers(listNum); // ok  
List<Object> listObj = new ArrayList<>();  
addNumbers(listObj); // ok  
List<Double> listDouble = new ArrayList<>();  
addNumbers(listDouble); // compile-time error, Double is not supertype of Integer
```

Wildcard (džoker) – izvođenje

- Generički tipovi nisu u relaciji izvođenja (nasleđivanja) samo zato što su u takvoj relaciji njihovi stvarni parametar tipovi.

```
class Base { /* ... */ }
```

```
class Derived extends Base { /* ... */ }
```

```
Derived d = new Derived();
```

```
Base b = d; // ok, is-a relationship
```

```
List<Derived> listDerived = new ArrayList<>();
```

```
List<Base> listBase = listDerived; // compile-time error
```

Wildcard (džoker) – izvođenje

- Iako je Integer podtip tipa Number, generički tip List<Integer> nije podtip tipa List<Number> i ova dva tipa podataka nisu ni u kakvoj vezi.
- Zajednički tip generičkih tipova List<Number> i List<Integer> je List<?>.
- Tip List<?> može da se oslanja samo na funkcionalnosti obezbeđene od strane klase Object.
- Da bi se u generičkim tipovima stvorila relacija između tipova Number i Integer koristi se gornje ograničeni džoker.

```
List<? extends Integer> intList = new ArrayList<>(); // list of Integers
```

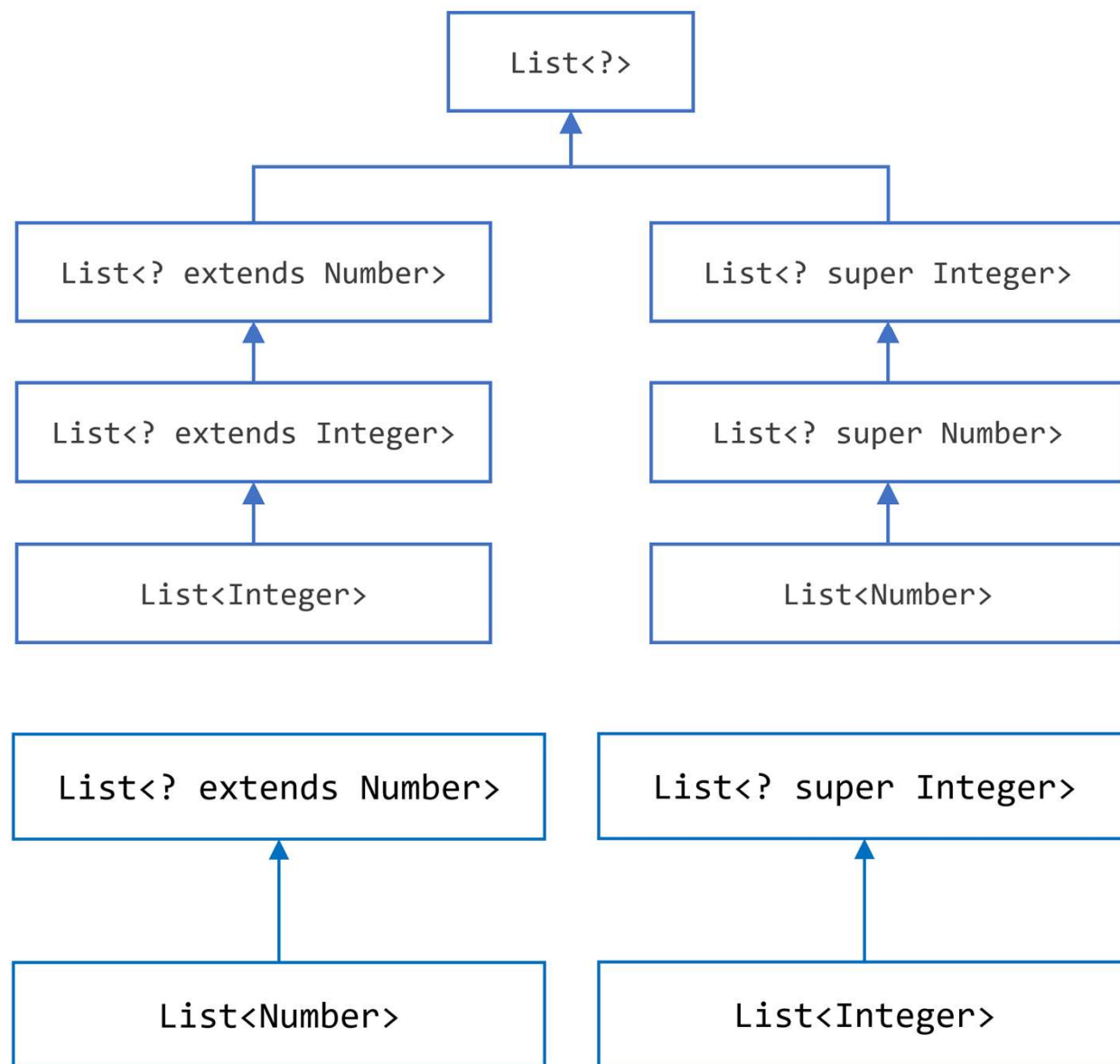
```
// line below is ok, since numList can refer to list of Numbers or its subtypes
```

```
List<? extends Number> numList = intList;
```

- Tip List<? extends Integer> je podtip tipa List<? extends Number>.
- Metode klase Number mogu da se koriste nad listom elemenata tipa Integer.

Wildcard (džoker) – izvođenje

- Neka je B nadtip tipa D.
- Generički tip `G<? extends B>` je nadtip tipa `G<? extends D>`, jer je skup tipova koji opisuje **? extends B** nadskup skupa tipova koje opisuje **? extends D**.
- Generički tip `G<? super D>` je nadtip tipa `G<? super B>`, jer je skup tipova koji opisuje **? super D** nadskup skupa tipova koje opisuje **? super B**.
- Bez džoker znaka `?`, `G<B>` i `G<D>` su invarijantni i nemaju nikakvu hijerarhijsku vezu, iako je tip B nadtip tipa D.



Wildcard capture (naprednije)

- U nekim situacijama prevodilac može, kada evaluiira izraz napisan u kodu, da zaključi šta je džoker parametar tip, što nije situacija u sledećem kodu.

```
public class Wildcard {  
    void foo(List<?> list) { list.set(0, list.get(0)); }  
    // compile-time error: incompatible types:  
    // Object cannot be converted to CAP#1 -> list.set(0, list.get(0));  
}
```

- Formalni parametar list je tipa List<CAP#1>, gde je CAP#1 neki nepoznati interni džoker tip.
- Prevodilac ne zna tačno koji je tip CAP#1, ali zna da CAP#1 extends Object (kao i svaki tip).
- Poziv list.get(0) vraća CAP#1 i, s obzirom da prevodilac ne može da potvrdi koji je to tip, jedino što je sigurno je da ga tretira kao tip Object (Object obj = list.get(0)).
- Poziv list.set(0, E element) očekuje za E tip CAP#1, dok poziv list.get(0) vraća nešto što se tretira kao Object i zato se javlja greška u prevođenju.
- Generici forsiraju tipnu sigurnost u vreme prevođenja programa.

Wildcard capture (naprednije)

- Pisanjem privatnog pomoćnog metoda može da se prevaziđe prethodno navedeni problem.

```
public class Wildcard {  
    // Helper method created  
    // so that the wildcard can be captured through type inference.  
    private <T> void fooHelper(List<T> list) { list.set(0, list.get(0)); }  
    void foo(List<?> list) { fooHelper(list); }  
}
```

- Parametar tip T u fooHelper metodi vrši wildcard capture, odnosno prevodilac zaključuje i vezuje interni tip CAP#1 za parametar tip T.
- Metode get i set liste u metodi fooHelper koriste isti parametar tip T, pa prevodilac može da potvrdi da su tipovi kompatibilni.

Wildcard capture (naprednije)

```
public class WildcardError {  
    void swapFirst(List<? extends Number> list1, List<? extends Number> list2) {  
        Number temp = list1.get(0); // ok, returns an upper bound type Number  
        // compile-time error: expected a CAP#1 extends Number, got a CAP#2 extends Number;  
        // same bound, but different types, since wildcards can be different types  
        list1.set(0, list2.get(0));  
        list2.set(0, temp); // compile-time error: expected a CAP#2 extends Number, got a Number  
    }  
}
```

- Kada bi se prethodno navedeni kod prevodio, sledeći kod bi bio ispravan, što ne bi bilo dobro, jer se narušava tipna sigurnost.

```
List<Integer> listInt = new ArrayList<>(Arrays.asList(1, 2, 3));
```

```
List<Double> listDouble = new ArrayList<>(Arrays.asList(10.11, 20.22, 30.33));
```

```
swapFirst(listInt, listDouble);
```

- Tada bi bilo validno da se Double podatak ubaci u listu Integer objekata i Integer podatak u listu Double objekata.

Wildcard capture (naprednije)

- Problem ne bi mogao da se prevaziđe ni pisanjem pomoćnog metoda kao u prethodnom slučaju, jer je kod fundamentalno neispravan i narušava tipnu sigurnost.

```
public class WildcardError {  
    <T extends Number> void swapFirstHelper(List<T> list1, List<T> list2) {  
        T temp = list1.get(0); list1.set(0, list2.get(0)); list2.set(0, temp);  
    }  
    void swapFirst(List<? extends Number> list1, List<? extends Number> list2) { swapFirstHelper(list1, list2); }  
}  
  
// compile-time error: required: List<T>,List<T> | found: List<CAP#1>,List<CAP#2>  
// reason: inference variable T has incompatible equality constraints CAP#2,CAP#1,  
// where T is a type-variable: T extends Number declared in method <T>swapFirstHelper(List<T>,List<T>),  
// where CAP#1,CAP#2 are fresh type-variables:  
//   CAP#1 extends Number from capture of ? extends Number  
//   CAP#2 extends Number from capture of ? extends Number
```

- Parametar tip T liste list1 je <? extends Number>, odnosno CAP#1 extends Number, dok parametar tip T liste list2 jeste <? extends Number>, odnosno CAP#2 extends Number, što su različiti tipovi, a parametar tip T treba da uradi wildcard capture za jedan isti tip.

Producer extends, Consumer super (PECS)

- Ukoliko generički tip samo proizvodi podatke (uzimanje iz generičke kolekcije), onda za parametar tip treba da se koristi **? extends T**, gde je T neki konkretan tip.
- Iz `List<? extends Number>` može da se čita samo `Number`, ali da se dodaje može samo `null`.
 - Jer takav generik može da referiše `List<Number>`, `List<Integer>`, `List<Double>` itd.

```
List<? extends Number> list = new ArrayList<>(Arrays.asList(1, 2, 3));
```

```
Number n = list.get(0); // ok
```

```
// list.add(4); // compile-time error
```

- Ukoliko generički tip samo prihvata podatke (dodavanje u generičku kolekciju), onda za parametar tip treba da se koristi **? super T**, gde je T neki konkretan tip.
- U `List<? super Integer>` može da se doda samo `Integer` ili podtip `Integer`, ali da se čita može samo `Object`.
 - Jer takav generik može da referiše `List<Integer>`, `List<Number>`, `List<Object>`.

```
List<? super Integer> list = new ArrayList<Number>();
```

```
list.add(10); // ok
```

```
Object o = list.get(0); // ok
```

Producer extends, Consumer super (PECS)

- Ukoliko generički tip *i* proizvodi podatke (uzimanje iz generičke kolekcije) i prihvata podatke (dodavanje u generičku kolekciju), onda treba da se koristi konkretan tip *T* bez džokera *?*.

```
List<Integer> list = new ArrayList<>();
```

```
list.add(11); // ok
```

```
Integer x = list.get(0); // ok
```

- Ukoliko se elementi koje generička kolekcija drži žele tretirati samo kao da su *Object* tipa (koriste se samo metode klase *Object*), onda treba da se koristi neograničen tip (samo džoker *?*).

```
public static void printList(List<?> lista) {  
    for (Object o : lista) { System.out.print(o); }  
}
```

```
printList(Arrays.asList(1, 2, 3)); // ok
```

```
printList(Arrays.asList("a", "b")); // ok
```

Producer extends, Consumer super (PECS)

- Nije preporučljivo da se generici sa ograničenim tipom koriste kao povratne vrednosti metoda jer ograničavaju šta može da radi kod pozivaoca i dodatno ga komplikuju.

```
public List<? extends Number> getList() {  
    return new ArrayList<>(Arrays.asList(1, 2, 3));  
}  
// forces the caller code to use wildcards  
List<? extends Number> list = getList();  
// restricts the caller to read-only access (no adding allowed)  
// list.add(10); // compile-time error  
list.get(0); // ok
```

Brisanje tipova (Type Erasure)

- Generici u Javi su uvedeni radi realizacije generičkih zbirki i algoritama, kao i da obezbede tipnu sigurnost i strožu proveru tipova u prilikom prevođenja programa.
- Java prevodilac implementira generike primenom koncepta brisanja tipova:
 - Zamenjuje sve parametar tipove ograničene gornjom granicom tipa njihovom gornjom granicom, a tipom Object, ukoliko su granice tipa donje ili nisu definisane.
 - `<T extends Number>` menja sa `Number`, `<? extends Number>` menja sa `Number`.
 - `<? super Integer>` menja sa `Object`, `<T>` menja sa `Object`.
 - Ukoliko je neophodno, umeće u bajtkod eksplicitnu bajtkod instrukciju **checkcast** da obezbedi tipnu sigurnost.
 - Instrukcija `checkcast` verifikuje da je referenca na vrhu steka specifičnog tipa (emituje se `ClassCastException` u suprotnom).
 - ```
class Box<T> {
 private T value;
 public T get() { return value; }
}
```

```
Box<String> box = new Box<>();
String s = box.get(); // box.get() method returns Object and compiler inserts explicit bytecode checkcast instruction
```
    - ```
List<? super Integer> list = new ArrayList<Number>();  
list.add(11); // no need for cast, since compiler knows 11 can be assigned to supertype of Integer
```
 - Generiše vezivne metode (bridge methods) da obezbedi polimorfizam u slučaju nasleđivanja.

Brisanje tipova (Type Erasure)

- Brisanjem tipova ne generišu se nove klase ili interfejsi na osnovu generičkih tipova za svaku njihovu parametrizaciju stvarnim parametar tipom.
- Ceo posao se obavlja u prilikom prevođenja programa.
 - Nema dodatnog runtime overhead-a.
- Rezultujući bajtkod sadrži samo prevedene obične Java tipove (klase i interfejse) i metode.

Brisanje tipova (Type Erasure)

- Realizacija generičke klase Node, koja predstavlja čvor jednostruko ulančane liste.

```
public class Node<T> {  
    private T data;  
    private Node<T> next;  
    public Node(T data, Node<T> next) { this.data = data; this.next = next; }  
    public T getData() { return data; }  
    // ...  
}
```

- S obzirom da parametar tip T nije ograničen, prevodilac ga menja tipom Object.

```
public class Node {  
    private Object data;  
    private Node next;  
    public Node(Object data, Node next) { this.data = data; this.next = next; }  
    public Object getData() { return data; }  
    // ...  
}
```

Brisanje tipova (Type Erasure)

- Realizacija generičke klase Node, koja predstavlja čvor jednostruko ulančane liste, uz gornje ograničeni parametar.

```
public class Node<T extends Comparable<T>> {  
    private T data;  
    private Node<T> next;  
    public Node(T data, Node<T> next) { this.data = data; this.next = next; }  
    public T getData() { return data; }  
    // ...  
}
```

- S obzirom da je parametar tip T ograničen, prevodilac ga menja tipom gornje granice, odnosno Comparable.

```
public class Node {  
    private Comparable data;  
    private Node next;  
    public Node(Comparable data, Node next) { this.data = data; this.next = next; }  
    public Comparable getData() { return data; }  
    // ...  
}
```

Brisanje tipova i generičke metode

- Previlac vrši brisanje parametar tipova i u generičkim metodama.

```
public static <T> int count(T[] anArray, T elem) {  
    int cnt = 0;  
    for (T e : anArray) if (e.equals(elem)) cnt ++;  
    return cnt;  
}
```

- S obzirom da parametar tip T nije ograničen, prevodilac ga menja tipom Object.

```
public static int count(Object[] anArray, Object elem) {  
    int cnt = 0;  
    for (Object e : anArray) if (e.equals(elem)) cnt ++;  
    return cnt;  
}
```

Brisanje tipova i generičke metode

- U generičkim metodama prevodilac menja parametar tipove ograničene gornjom granicom tipa njihovom gornjom granicom tipa, a tipom Object, ukoliko su granice tipa donje ili nisu definisane.

- Ukoliko su definisane sledeće klase:

```
class Shape { /* ... */ }
```

```
class Circle extends Shape { /* ... */ }
```

```
class Rectangle extends Shape { /* ... */ }
```

- i generički metod koji iscrtava različite oblike

```
public static <T extends Shape> void draw(T shape) { /* ... */ }
```

- prevodilac vrši zamenu parametar tipa T gornjom granicom tipa Shape.

```
public static void draw(Shape shape) { /* ... */ }
```

Brisanje tipova i vezivne metode

- Ponekad, kada prevodilac prevodi klasu ili interfejs koji proširuju generičku klasu ili implementiraju generički interfejs, prevodilac kreira sintetički vezivni metod (bridge method) kao deo procesa brisanja tipova.

```
public class Node<T> {  
    public T data;  
    public Node(T data) { this.data = data; }  
    public void setData(T data) { this.data = data; }  
}  
  
public class MyNode extends Node<Integer> {  
    public MyNode(Integer data) { super(data); }  
    @Override  
    public void setData(Integer data) { super.setData(data); }  
}
```

Brisanje tipova i vezivne metode

- Nakon primene brisanja tipova od strane prevodioca, prethodni kod postaje:

```
public class Node {  
    public Object data;  
    public Node(Object data) { this.data = data; }  
    public void setData(Object data) { this.data = data; }  
}  
  
public class MyNode extends Node {  
    public MyNode(Integer data) { super(data); }  
    public void setData(Integer data) { super.setData(data); }  
}
```

- Potpisi nestatičkih metoda `Node.setData(Object)` (ranije `Node.setData(T)`) i `MyNode.setData(Integer)` više nisu odgovarajući.
- Metod `MyNode.setData(Integer)` ne predstavlja redefiniciju metoda `Node.setData(Object)`, već je u pitanju preklapanje imena metoda (method overloading).

Brisanje tipova i vezivne metode

- Da bi se prevazišao prethodni problem i očuvao polimorfizam generičkih tipova nakon procesa brisanja tipova, prevodilac generiše vezivne metode da osigura da se nasleđivanje ponaša očekivano.

```
class MyNode extends Node {  
    // Bridge method generated by the compiler (visible in JVM, but hidden from the programmer)  
    public void setData(Object data) { setData((Integer) data); } // verifying argument is of type Integer  
  
    public MyNode(Integer data) { super(data); }  
    public void setData(Integer data) { super.setData(data); }  
}  
  
MyNode mn = new MyNode(5);  
Node n = mn; // raw type used - compiler throws an unchecked warning  
// Raw type bypasses generic type checking.  
// Due to polymorphism, the compiler-generated bridge method MyNode.setData(Object) is invoked.  
n.setData(11); // ok  
n.setData("Hello"); // causes a ClassCastException to be thrown at runtime
```

Ograničenja generičkih tipova

- **Primitivni tipovi ne mogu biti stvarni parametar tipovi generika.**

```
class Pair<K, V> {  
    private K key;  
    private V value;  
    public Pair(K key, V value) { this.key = key; this.value = value; }  
    // ...  
}
```

```
Pair<int, char> p = new Pair<>(8, 'a'); // compile-time error
```

```
// Java compiler autoboxes 8 to Integer.valueOf(8) and 'a' to Character('a')
```

```
Pair<Integer, Character> p = new Pair<>(8, 'a'); // ok
```

```
Pair<Integer, Character> p = new Pair<>(Integer.valueOf(8), new Character('a')); // ok
```

Ograničenja generičkih tipova

- **Ne mogu da se kreiraju instance navođenjem formalnih parametar tipova.**

```
public static <E> void append(List<E> list) {  
    E elem = new E(); // compile-time error  
    list.add(elem);  
}
```

- Može da se koristi refleksija da zaobiđe ova ograničenja.

```
public static <E> void append(List<E> list, Class<E> cls) throws Exception {  
    E elem = cls.newInstance(); // ok  
    list.add(elem);  
}
```

```
List<String> listStr = new ArrayList<>();  
append(listStr, String.class);
```

Ograničenja generičkih tipova

- **Ne mogu da se definišu statička polja čiji su tipovi formalni parametar tipovi generika ili da se takvi parametar tipovi koriste u statičkim metodama.**

```
public class MobileDevice<T> {  
    private static T os; // compile-time error  
    public static T getOs() { return os; } // compile-time error  
    // ...  
}
```

- Parametar tipovi generika su nakon procesa prevođenja i brisanja tipova obrisani.

```
MobileDevice<Smartphone> phone = new MobileDevice<>();
```

```
MobileDevice<Pager> pager = new MobileDevice<>();
```

```
MobileDevice<TabletPC> pc = new MobileDevice<>();
```

- U vreme izvršavanja programa postoji samo jedna klasa MobileDevice.
- Kog tipa bi onda bilo statičko polje MobileDevice.os? (retoričko pitanje)

Ograničenja generičkih tipova

- Da je takvo statičko polje dozvoljeno, nakon brisanja tipova njegov generički formalni parametar tip bio bi zamenjen gornjom granicom tipa (ako je ona definisana) ili tipom Object.
- S obzirom da je statičko polje deljeno među objektima klase, svaki objekat generika bi mogao da ga postavlja na podatke različitih stvarnih parametar tipova generika.
 - Što bi bilo u redu sa sintaksnog stanovišta, ali ne i sa semantičkog.
- Međutim, kod dohvatanja bi došlo do pokušaja konverzije tipa u stvarni parametar tip zbog tipne sigurnosti, što bi dovelo do grešaka u vreme izvršavanja programa (ClassCastException).
- Sa druge strane, nestatička polja čiji su tipovi formalni parametar tipovi generika su dozvoljena, jer svaka instanca poseduje svoje polje.
Takvi generički formalni parametar tipovi mogu da se koriste i u nestatičkim metodama.
- Statički članovi pripadaju klasi, dok je generički formalni parametar tip vezan za instancu.
- Statički metod može definisati sopstveni generički formalni parametar tip (to nije generički formalni parametar tip klase ili interfejsa):

```
class Printer<T> {  
    public static <S> void print(S value) { } // ok  
}
```

Ograničenja generičkih tipova

- **Generički tip parametrizovan stvarnim parametar tipom ne može da se koristi uz operator instanceof.**
- S obzirom da prevodilac briše sve formalne parametar tipove generika, ne bi ni mogla da se napravi razlika između npr. `ArrayList<Integer>` i `ArrayList<String>`.

```
public static <E> void rtti(List<E> list) {  
    if (list instanceof ArrayList<Integer>) { // compile-time error  
    }  
}
```

- Može da se koristi neograničeni džoker parametar tip generika da se ispita da li je instanca određene generičke klase bez obzira na parametar tipa.

```
public static void rtti(List<?> list) {  
    if (list instanceof ArrayList<?>) { // ok; same as writing ArrayList, which is legacy raw-type  
    }  
}
```

Ograničenja generičkih tipova

- **Konverzija (cast) između parametrizovanih generičkih tipova nije dozvoljena, ukoliko prevodilac može da zaključi da su tipovi nekompatibilni.**

```
List<Integer> listInt = new ArrayList<>();
```

```
// incompatible types in hierarchy: List<Integer> is not a subtype of List<Number>
```

```
List<Number> listNum1 = listInt; // compile-time error, implicit conversion tried
```

```
List<Number> listNum2 = (List<Number>) listInt; // compile-time error, explicit conversion tried
```

```
List<? extends Number> listNumExt = (List<? extends Number>) listInt; // ok, cast not needed
```

```
List<?> listUnb = (List<?>) listInt; // ok, cast not needed
```

```
List<Number> listNum = new ArrayList<>();
```

```
List<? super Integer> listIntSup = (List<? super Integer>) listNum; // ok, cast not needed
```

```
// compatible types in hierarchy: ArrayList<String> is a subtype of List<String>
```

```
List<String> listStr = new ArrayList<>();
```

```
ArrayList<String> arrayListStr = (ArrayList<String>) listStr; // ok, cast is mandatory!
```

Ograničenja generičkih tipova

- **Ne mogu da se kreiraju nizovi objekata čiji su elementi parametrizovani generički tipovi.**

```
List<Integer>[] arrayOfLists = new List<Integer>[2]; // compile-time error
```

- Nizovi treba da budu sekvence homogenih elemenata određenog tipa ili njegovih podtipova.

```
Number arr[] = new Integer[3];
```

```
arr[0] = 12; // ok, Integer is a subtype of Number
```

```
arr[1] = 3.14; // runtime-error: ArrayStoreException – Double is Number, but not Integer
```

```
arr[2] = "hi"; // compile-time error: String is not a subtype of Number
```

```
Object[] strings = new List<String>[2]; // compile-time error, but assume it is allowed (hypothetical model)
```

```
// line below would be allowed in compile-time, since ArrayList<String> is a subtype of Object,  
// and would be ok in runtime, since ArrayList is a subtype of List (there are no parameter types in runtime)
```

```
strings[0] = new ArrayList<String>();
```

```
// line below would be allowed in compile-time, since ArrayList<Integer> is a subtype of Object,  
// but unfortunately, ArrayStoreException could not be thrown in runtime (since ArrayList is a subtype of List)
```

```
strings[1] = new ArrayList<Integer>();
```

Ograničenja generičkih tipova

- **Generičke klase ne mogu direktno ili indirektno da proširuju klasu Throwable.**

// Generic class cannot extend Throwable directly

```
class QueueFullException<T> extends Throwable { /* ... */ } // compile-time error: Exceptions must be differentiated in runtime
```

// Generic class cannot extend Throwable indirectly

```
class MathException<T> extends Exception { /* ... */ } // compile-time error: same as above
```

- **Parametar tip generika ne može da bude parametar catch klauzule.**

```
public static <T extends Exception, J> void execute(List<J> jobs) {  
    try { // processing list of jobs ...  
        } catch (T e) { // compile-time error: exception type must be known in runtime, but type erasure occurred  
        }  
}
```

- Ipak, parametar tip generika može da bude deo throws klauzule.

```
class Parser<T extends Exception> {  
    public void parse(File file) throws T { } // allowed: checked at compile time  
}
```

Ograničenja generičkih tipova

- Tipovi ne mogu da definišu dva ili više metoda preklopljenih imena (name overloading) čiji će potpisi biti ekvivalentni nakon brisanja tipova (type erasure).
- Ukoliko se to pokušava ostvariti, grešku će prijaviti prevodilac.

```
public class Example {  
    // both methods have signatures foo(Set s) after type-erasure  
    public void foo(Set<String> strSet) { }  
    public void foo(Set<Integer> intSet) { }  
}
```

Poznati generički tipovi

- Navedeni interfejsi su funkcionalni interfejsi i često se koriste zajedno sa lambda izrazima.

- **Predicate<T>** - Provera uslova.

```
interface Predicate<T> { boolean test(T t); }  
Predicate<Integer> isEven = x -> x % 2 == 0;
```

- **Consumer<T>** - Obrada objekta.

```
interface Consumer<T> { void accept(T t); }  
Consumer<String> printer = s -> System.out.println(s);
```

- **Supplier<T>** - Generisanje ili vraćanje objekta.

```
interface Supplier<T> { T get(); }  
Supplier<Double> random = () -> Math.random();
```

- **Function<T, R>** - Transformacija objekta.

```
interface Function<T, R> { R apply(T t); }  
Function<String, Integer> length = s -> s.length();
```

Poznati generički tipovi

- **List<T>** - Uređena kolekcija koja čuva redosled elemenata i dozvoljava duplikate. Omogućava indeksirani pristup elementima.
- **ArrayList<T>** - Implementacija liste bazirana na dinamičkom nizu. Brz pristup elementima po indeksu, ali sporije ubacivanje/brisanje u sredini.
- **LinkedList<T>** - Implementacija liste bazirana na dvostruko povezanoj strukturi. Efikasna za često ubacivanje i brisanje elemenata, ali sporiji random access.

```
List<Integer> arr = new ArrayList<>();  
arr.add(10);  
System.out.println(arr.size()); // prints: 1  
arr.get(0);  
arr.remove(0);
```

```
List<Integer> list = new LinkedList<>();  
list.add(0, 123);  
list.clear(); // removes all elements and resets size to 0
```

Poznati generički tipovi

- **Set<T>** - Kolekcija jedinstvenih elemenata bez duplikata.
Redosled nije garantovan (osim kod specifičnih implementacija).
- **HashSet<T>** - Kolekcija jedinstvenih elemenata bez garantovanog redosleda.
Najbrža osnovna implementacija seta.
- **LinkedHashSet<T>** - Set koji čuva redosled ubacivanja elemenata.
Kombinuje jedinstvenost i predvidiv redosled.
Sporiji od HashSet-a.

```
Set<Integer> hs = new HashSet<>();  
hs.add(1);  
hs.add(1); // ignored  
System.out.println(hs.contains(1)); // prints: true
```

```
Set<String> lhs = new LinkedHashSet<>();  
lhs.add("A");  
lhs.remove("A");  
System.out.println(lhs.isEmpty()); // prints: true
```

Poznati generički tipovi

- **Map<K, V>** - Struktura koja mapira ključeve na vrednosti. Ključevi su jedinstveni i koriste se za brzo pretraživanje.
- **HashMap<K, V>** - Najosnovnija implementacija mape bez garantovanog redosleda. Omogućava brze operacije ubacivanja i pristupa.
- **LinkedHashMap<K, V>** - Mapa koja čuva redosled ubacivanja (ili pristupa, ako se podesi). Koristi se kada je bitan redosled iteracije.
- **TreeMap<K, V>** - Sortirana mapa koja održava ključeve u prirodnom redosledu ili po comparator-u.

```
Map<String, Integer> hm = new HashMap<>();  
hm.put("A", 1);  
System.out.println(hm.get("A")); // prints: 1
```

```
Map<String, Integer> lhm = new LinkedHashMap<>();  
lhm.put("B", 2);  
lhm.remove("B");
```

```
Map<Integer, String> tm = new TreeMap<>();  
tm.put(3, "C");
```

Poznati generički tipovi

- **Queue<T>** - FIFO struktura (First In First Out). Elementi se obrađuju redosledom dodavanja.
- **Deque<T>** - Dvostrani red koji omogućava rad na oba kraja (stack + queue).
- **ArrayDeque<T>** - Implementacija Deque<T> interfejsa koja omogućava efikasan rad sa oba kraja reda.

```
Queue<Integer> q = new LinkedList<>();
q.add(1);
int val = q.peek(); // returns first element without removing (or null, if queue is empty)
// q.element() alternative to q.peek(), which throws NoSuchElementException if queue is empty
val = q.poll(); // removes and returns first element (or null, if queue is empty)
// q.remove() alternative to q.poll(), which throws NoSuchElementException if queue is empty

Deque<Integer> dq = new ArrayDeque<>();
dq.addFirst(1);
dq.addLast(2);

ArrayDeque<String> deque = new ArrayDeque<>();
deque.addFirst("A");
deque.addLast("B");
System.out.println(deque.removeFirst()); // prints: A
System.out.println(deque.removeLast()); // prints: B
```

Poznati generički tipovi

- **Optional<T>** - Kontejner koji može da sadrži ili ne sadrži vrednost. Koristi se da se izbegne null.

```
Optional<String> opt = Optional.of("John");  
System.out.println(opt.isPresent()); // prints: true
```

```
Optional<String> empty = Optional.empty();  
System.out.println(empty.orElse("Unknown")); // prints: Unknown
```

- **Comparable<T>** - Definiše prirodni poredak objekata i omogućava njihovo sortiranje.

```
class Person implements Comparable<Person> {  
    public int compareTo(Person p) {  
        // should return a negative value, 0, or a positive value  
        // based on comparison  
    }  
}
```

Literatura

- **dev.java/learn** - Oracle official Java learning platform; početnički nivo.
- **Head First Java** - **Kathy Sierra** (Java edukator), **Bert Bates** (Java instruktor); vizuelno učenje i analogije; početnički nivo.
- **Core Java** - **Cay Horstmann** (programer, profesor i Java autoritet); detaljno jezgro jezika Java; srednji nivo.
- **Java: The Complete Reference** - **Herbert Schildt** (programer i tehnički autor); referenca i kompletan pregled jezika Java; srednji nivo.
- **Effective Java** - **Joshua Bloch** (Java inženjer); najbolje prakse jezika Java; napredni nivo.